

Exercise 6

Strings

For this exercise, you're going to write a program that performs a given sequence of basic operations on a string, inserting and deleting characters at given locations. It will read the sequence of string operations from standard input. The program will be named `editstring.c`. You can download a starter for this program and supporting files from the course Moodle page, or, if you're logged in to `remote.eos.ncsu.edu`, you can use the following shell command to copy the needed files to the current directory:

```
cp /mnt/coe/workspace/csc/CSC230/ex/exercise06/* .
```

String Editing

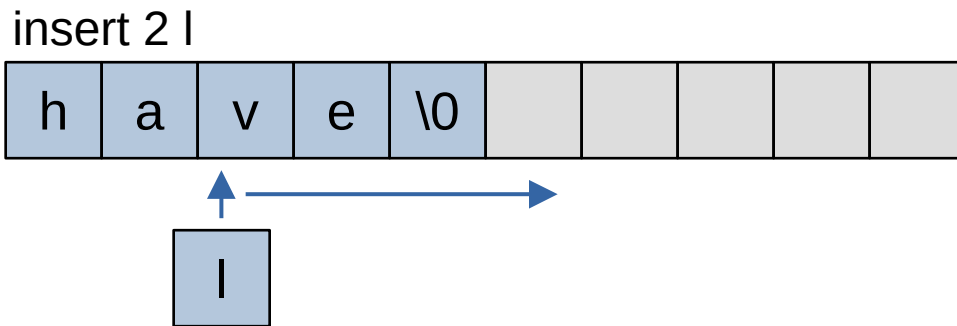
The program will start with an empty string. The string will be modified based on edit operations read from standard input. There are two types of edit operations, inserting a character anywhere in the string or removing a character from anywhere in the string:

- `insert p ch`

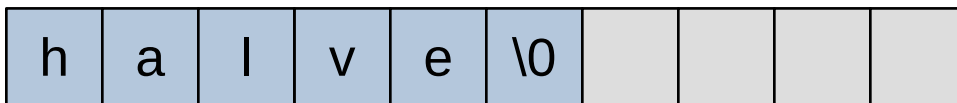
The insert operation puts the given character, *ch*, at index, *p*, in the string. All characters previously at position *p* or later in the string are moved to the right. The position counts from 0 for the first character up to and including the current length of the string. For example, if you had a string of length 4, inserting a character at position 0 would put it at the front (moving all other characters to the right) and inserting at position 4 would put the new character at the end.

The character, *ch*, can be any printable, non-whitespace character, anything with ASCII code 0x21 up to 0x7E. The position, *p*, can be anything from zero up to the previous string length. Insert is only valid if the string length is less than 80 (so, the string can never become longer than 80 characters). If the insert command doesn't satisfy these constraints, the program should print the line "Invalid input" to the terminal and exit with a status of 1. The starter has a function, `invalid()`, to do this for you.

The following shows inserting the letter l at position 2 in the 4-character string, "have":



After inserting this letter, the string contains "halve":

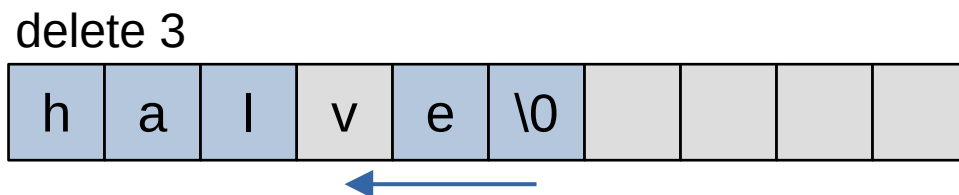


- delete p

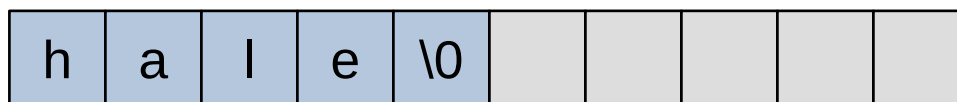
The delete operation removes the character from position p in the string, moving any characters after p to the left by one. The value of p can be the index of any character in the string, counting from zero as the first character. So, in a string of length 4, p can be any value from 0 up to 3.

A delete command is invalid if the given position is out of range or if the string is already empty. Invalid delete commands are handled just like invalid inserts; the program should print the line "Invalid input" to the terminal and exit with a status of 1.

The following figure shows a delete of the character at position 3 in a 5 character string:



After deleting this character, the string will look like:



Edit operations and their parameters can be separated by any amount of whitespace. We'll always give each operation on an input line by itself, but it's OK if your program accepts operations even if they are split across multiple lines (scanf() makes it easier to parse input

like this). If the program is given an invalid operation (anything other than a valid insert or delete), it should print a line “Invalid input” to the terminal and exit with a status of 1.

After each edit operation, the program will print out a line (to standard output) giving the current contents of the string. This will let you watch how the string changes as edits are applied.

Sample Execution

Once your program is working, you should be able to run it against the sample inputs provided, or you can try making a test input yourself. The following example shows running the program with edit operations read from the input-1.txt file. The box on the left shows the user running the program, reading input from the input-1.txt file. This is followed by the program’s output. The box on the right shows the contents of the input-1.txt file, to make it easy to see what edits are being applied to the string.

terminal	input-1.txt
<pre>\$./editstring < input-1.txt x xc xbc xabc abc</pre>	<pre>insert 0 x insert 1 c insert 1 b insert 1 a delete 0</pre>

There are two other sample inputs, with expected output files. You can run these like the following to help make sure your program is behaving right. Here, we’re running the program with input redirected from one of the sample input files and output captured in output.txt. Immediately after execution, we check the exit status, to make sure the program ran successfully. It should exit successfully for test inputs 1 and 2, but it should exit with an error on input 3. Finally, we check the program output to make sure it matches the expected output.

terminal
<pre>\$./editstring < input-1.txt > output.txt \$ echo \$? 0 \$ diff output.txt expected-1.txt</pre>

The input-3.txt file contains an error test. It tries to make a string that's too long. When you run it, you should eventually get "Invalid input" printed to the terminal and the program should exit with an exit status of 1.

terminal

```
$ ./editstring < input-3.txt
A
BA
CBA
... some output omitted ...
AZYXWVUTSRQPONMLKJIHGFEDCBAZYXWVUTSRQPONMLKJIHGFEDCBA
BAZYXWVUTSRQPONMLKJIHGFEDCBAZYXWVUTSRQPONMLKJIHGFEDCBA
Invalid input
$ echo $?
1
```

Submitting Your Work

When you're done, submit your completed editstring.c file to the exercise_06 assignment in Moodle.